

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №1

по дисциплине: Тестирование программных систем.

тема: «Составление тест-кейсов для готового кода. Качество ПО и место
тестирования в жизненном цикле»

Выполнил: ст. группы ПВ-233

Ситников Алексей Павлович

Проверил:

Доденко Олег Павлович

Белгород 2026 г.

Цель работы: сформировать системное представление о качестве ПО и роли тестирования в жизненном цикле; приобрести практические навыки проектирования тест-кейсов и написания автотестов на основе готового кода.

Задачи:

1. Проанализировать готовый модуль и выделить бизнес-правила, входные и выходные данные, ограничения.
2. Составить набор тест-кейсов: позитивных, негативных, граничных, покрывающих эквивалентные классы.
3. Оформить тест-кейсы по стандартным атрибутам.
4. Реализовать автотесты на Python + pytest.
5. Запустить тесты и зафиксировать результаты.
6. Сделать вывод о связи тестирования, качества ПО и жизненного цикла.

Исходный код:

```
def calculate_order_total(items_total: float, distance_km: float, is_premium:
bool) -> float:
    """
    Расчёт итоговой стоимости заказа с учётом доставки.

    Бизнес-правила:
    1. Если items_total < 0 или distance_km < 0 - ValueError.
    2. Если items_total >= 3000 - доставка бесплатна.
    3. Иначе стоимость доставки:
        - distance_km <= 3: 150
        - 3 < distance_km <= 10: 300
        - distance_km > 10: 300 + (distance_km - 10) * 25
    4. Для premium-пользователя скидка 20% на стоимость доставки.
    5. Итоговая сумма = items_total + стоимость доставки.
    """
    if items_total < 0:
        raise ValueError("items_total must be >= 0")
    if distance_km < 0:
        raise ValueError("distance_km must be >= 0")

    if items_total >= 3000:
        delivery_cost = 0.0
    elif distance_km <= 3:
        delivery_cost = 150.0
    elif distance_km <= 10:
        delivery_cost = 300.0
    else:
        delivery_cost = 300.0 + (distance_km - 10) * 25.0

    if is_premium:
        delivery_cost *= 0.8

    return items_total + delivery_cost
```

- входные параметры:
 - items_total: float — сумма товаров в заказе;
 - distance_km: float — расстояние доставки в километрах;
 - is_premium: bool — является ли пользователь premium.
- Выходной параметр:
 - float — итоговая стоимость заказа с доставкой.
- Бизнес-правила:
 - items_total < 0 → ValueError.
 - distance_km < 0 → ValueError.
 - Если items_total ≥ 3000, доставка бесплатна.
 - Иначе стоимость доставки:
 - до 3 км включительно — 150;
 - от 3 до 10 км включительно — 300;
 - больше 10 км — 300 + (distance_km - 10) * 25.
 - Premium-пользователь получает скидку 20% на доставку.
 - Если доставка бесплатна, скидка не меняет результат: 0 * 0.8 = 0.
 - Итоговая стоимость = сумма товаров + доставка.
- Допустимые значения:
 - items_total ≥ 0;
 - distance_km ≥ 0;
 - is_premium: True или False.
- Недопустимые значения:
 - items_total < 0;
 - distance_km < 0.
- Граничные значения
 - items_total: 0, 2999.99, 3000, 3000.01;
 - distance_km: 0, 3, 3.01, 10, 10.01;
 - is_premium: False, True.

Тест кейсы:

№	Название	Тип	Входные данные	Ожидаемый результат	Проверяемое правило
1	Нулевая сумма и нулевое расстояние	Позитивный , граничный	items_total=0, distance_km=0, is_premium=False	150.0	Валидные нулевые значения, тариф до 3 км
2	Минимальная положительная сумма	Позитивный , граничный	0.01, 3, False	150.01	Тариф до 3 км
3	Граница 3 км	Граничный	1000, 3, False	1150.0	distance_km <= 3
4	Чуть больше 3 км	Граничный	1000, 3.01, False	1300.0	3 < distance_km <= 10
5	Граница 10 км	Граничный	1000, 10, False	1300.0	3 < distance_km <= 10
6	Чуть больше 10 км	Граничный	1000, 10.01, False	1300.25	distance_km > 10, формула
7	Длинная дистанция 20 км	Позитивный	1000, 20, False	1550.0	300 + (20 - 10) * 25 = 550
8	Сумма 2999.99	Граничный	2999.99, 10, False	3299.99	Доставка ещё платная
9	Сумма 3000	Граничный, бесплатная доставка	3000, 10, False	3000.0	Бесплатная доставка от 3000
10	Сумма 5000, длинная дистанция	Позитивный , бесплатная доставка	5000, 50, False	5000.0	Бесплатная доставка покрывает расчёт по км
11	Premium, средняя дистанция	Позитивный , premium	1000, 10, True	1240.0	Скидка 20% на 300 → 240
12	Premium, длинная дистанция	Позитивный , premium	1000, 20, True	1440.0	Скидка 20% на 550 → 440
13	Premium и бесплатная доставка	Граничный, premium	3000, 20, True	3000.0	Скидка не делает стоимость отрицательной
14	Отрицательная сумма	Негативный	-1, 10, False	ValueError	Валидация items_total
15	Отрицательное расстояние	Негативный	1000, -1, False	ValueError	Валидация distance_km

Автотесты:

```
import pytest

from main import calculate_order_total

def test_zero_items_and_zero_distance():
    assert calculate_order_total(0, 0, False) == pytest.approx(150.0)

def test_min_positive_items_short_distance():
    assert calculate_order_total(0.01, 3, False) == pytest.approx(150.01)

def test_distance_boundary_3_km():
    assert calculate_order_total(1000, 3, False) == pytest.approx(1150.0)

def test_just_over_3_km():
    assert calculate_order_total(1000, 3.01, False) == pytest.approx(1300.0)

def test_distance_boundary_10_km():
    assert calculate_order_total(1000, 10, False) == pytest.approx(1300.0)

def test_just_over_10_km():
    assert calculate_order_total(1000, 10.01, False) ==
pytest.approx(1300.25)

def test_long_distance_20_km():
    assert calculate_order_total(1000, 20, False) == pytest.approx(1550.0)

def test_items_below_free_threshold_2999_99():
    assert calculate_order_total(2999.99, 10, False) ==
pytest.approx(3299.99)

def test_free_delivery_at_3000():
    assert calculate_order_total(3000, 10, False) == pytest.approx(3000.0)

def test_free_delivery_above_3000_long_distance():
    assert calculate_order_total(5000, 50, False) == pytest.approx(5000.0)

def test_premium_discount_middle_distance():
    assert calculate_order_total(1000, 10, True) == pytest.approx(1240.0)

def test_premium_discount_long_distance():
    assert calculate_order_total(1000, 20, True) == pytest.approx(1440.0)

def test_premium_free_delivery():
    assert calculate_order_total(3000, 20, True) == pytest.approx(3000.0)

def test_negative_items_total():
    with pytest.raises(ValueError, match="items_total must be >= 0"):
```

```

        calculate_order_total(-1, 10, False)

def test_negative_distance():
    with pytest.raises(ValueError, match="distance_km must be >= 0"):
        calculate_order_total(1000, -1, False)

```

```

(.venv) PS C:\Users\alex\PycharmProjects\stlab1> pytest -v
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\alex\PycharmProjects\stlab1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\alex\PycharmProjects\stlab1
collected 15 items

test_delivery.py::test_zero_items_and_zero_distance PASSED [ 6%]
test_delivery.py::test_min_positive_items_short_distance PASSED [ 13%]
test_delivery.py::test_distance_boundary_3_km PASSED [ 20%]
test_delivery.py::test_just_over_3_km PASSED [ 26%]
test_delivery.py::test_distance_boundary_10_km PASSED [ 33%]
test_delivery.py::test_just_over_10_km PASSED [ 40%]
test_delivery.py::test_long_distance_20_km PASSED [ 46%]
test_delivery.py::test_items_below_free_threshold_2999_99 PASSED [ 53%]
test_delivery.py::test_free_delivery_at_3000 PASSED [ 60%]
test_delivery.py::test_free_delivery_above_3000_long_distance PASSED [ 66%]
test_delivery.py::test_premium_discount_middle_distance PASSED [ 73%]
test_delivery.py::test_premium_discount_long_distance PASSED [ 80%]
test_delivery.py::test_premium_free_delivery PASSED [ 86%]
test_delivery.py::test_negative_items_total PASSED [ 93%]
test_delivery.py::test_negative_distance PASSED [100%]

===== 15 passed in 0.04s =====

```

Все тесты пройдены.

Вывод: в ходе лабораторной работы был разработан собственный модуль, реализующий расчёт итоговой стоимости заказа с доставкой. Для него выделены бизнес-правила, входные и выходные параметры, допустимые, недопустимые и граничные значения.

Составлено 15 тест-кейсов: позитивные, негативные, граничные, проверяющие premium-скидку и бесплатную доставку. Написаны автотесты на pytest, которые успешно проходят.

Покрытые ветви кода:

- валидация отрицательной суммы заказа;
- валидация отрицательного расстояния;
- бесплатная доставка при сумме от 3000;
- тариф до 3 км;
- тариф от 3 до 10 км;
- тариф свыше 10 км;
- premium-скидка 20% на доставку;
- ситуация, когда premium-скидка применяется к бесплатной доставке.

Оставшиеся риски:

- не проверяются типы данных, NaN, inf, очень большие числа;
- не проверяется округление денежных значений;

Тестирование очень важный этап разработки программного продукта, так как оно помогает обнаружить дефекты на ранних стадиях разработки, когда исправления стоят минимальных затрат, а также помогает не пропускать неработающий по конкретной аналитике код в prod.